

Practical 7 (References)



Setting Permission

- For Android applications, permission is required for networking to work
- The permission can be added into the AndroidManifest.xml
- The following statements should be added to the manifest file:

```
<uses-permission  
    android:name="android.permission.INTERNET"/>
```

Reading Data – example

```
import java.io.InputStream;
import java.net.URL;

try {
    URL url = new URL("http://www.mad.edu.my/example.html");

    InputStream input = url.openStream();
    byte[] binput = new byte[1000];
    int readSize = input.read(binput);

    Toast.makeText(this, new String(binput),
        Toast.LENGTH_LONG).show();
    input.close();
} catch (Exception e) {
    Log.e("Net", "Error", e);
}
```

Reading Data – example

- Instead of reading into a fixed size buffer, `BufferedInputStream` can be used to read in data instead

```
URL url = new URL(" http://www.mad.edu.my/example.html");  
hc = (HttpURLConnection) url.openConnection();  
final String result;  
InputStream in = new BufferedInputStream(hc.getInputStream());  
result = readStream(in);
```

Reading Data – example

```
private String readStream(InputStream is) {  
    try {  
        ByteArrayOutputStream bo = new  
            ByteArrayOutputStream();  
        int i = is.read();  
        while (i != -1) {  
            bo.write(i);  
            i = is.read();  
        }  
        return bo.toString();  
    } catch (IOException e) {  
        return "";  
    }  
}
```

Thread – example

```
final Handler mHandler = new Handler();
```

```
final TextView tv = new TextView(this);
```

```
new Thread(){
```

```
    public void run(){
```

```
        try {
```

```
            URL url = new
```

```
                URL("http://api.flickr.com/services/feeds/  
photos_public.gne?id=26648248@N04&  
lang=en-us&format=atom");
```

```
            XmlPullParserFactory parserCreator =
```

```
                XmlPullParserFactory.newInstance();
```

```
            XmlPullParser parser =
```

```
                parserCreator.newPullParser();
```

Thread – example

```
parser.setInput(url.openStream(), null);
```

```
mHandler.post(new Runnable() {  
    public void run() {  
        tv.setText("Parsing...");  
    }  
});
```

```
int parserEvent = parser.getEventType();  
while (parserEvent != XmlPullParser.END_DOCUMENT) {  
    :  
    :  
    :  
    parserEvent = parser.next();  
}
```

Thread – example

```
mHandler.post(new Runnable() {  
    public void run() {  
        tv.setText("Done...");  
    }  
});  
  
} catch (Exception e) {  
    Log.e("Net", "Error", e);  
}  
  
}  
}.start();
```

MainActivity (GET – example)

```
package my.edu.utar.networkproject;

import java.io.InputStream;
import java.net.*;

import android.os.Bundle;
import android.os.Handler;
import android.text.InputType;
import android.util.Log;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.*;
import android.app.Activity;
import android.content.Intent;

public class MainActivity extends Activity {
    private static EditText et2;
```

MainActivity (GET – example)

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    final Handler mHandler = new Handler();

    TextView tv1 = new TextView(this);
    tv1.setText("Login ID:");
    TextView tv2 = new TextView(this);
    tv2.setText("Password:");
    final EditText et1 = new EditText(this);
    et2 = new EditText(this);
    et2.setInputType(InputType.TYPE_CLASS_TEXT |
        InputType.TYPE_TEXT_VARIATION_PASSWORD);
    Button bt = new Button(this);
    bt.setText("Login");

    LinearLayout ll = new LinearLayout(this);
    ll.setOrientation(LinearLayout.VERTICAL);
```

MainActivity (GET – example)

```
ll.addView(tv1);
ll.addView(et1);
ll.addView(tv2);
ll.addView(et2);
ll.addView(bt);
setContentView(ll);

bt.setOnClickListener(new OnClickListener() {
    @Override
    public void onClick(View v) {
        MyThread connectThread = new MyThread(
            et1.getText().toString(),
            et2.getText().toString(), mHandler);
        connectThread.start();
    }
});
}
```

MainActivity (GET – example)

```
private class MyThread extends Thread{
    private String name, pwd;
    private Handler mHandler;

    public MyThread(String name, String pwd, Handler handler){
        this.name = name;
        this.pwd = pwd;
        mHandler = handler;
    }

    public void run(){
        try {
            URL url = new URL(
                "http://10.0.2.2:8080/ServletTest/"+
                "LoginGet?name="+name+"&password="+pwd);

            Log.i("Net",url.toString());
        }
    }
}
```

MainActivity (GET – example)

```
HttpURLConnection hc =  
    (HttpURLConnection)url.openConnection();  
int length = hc.getContentLength();  
  
InputStream input = url.openStream();  
byte[] binput = new byte[length];  
int readSize = input.read(binput);  
  
Log.i("Net","readSize = " + readSize);  
Log.i("Net","bText = "+ new String(binput));  
  
String result = new String(binput).trim();  
  
if (result.equals("OK")){  
    Intent i = new Intent(MainActivity.this,  
        SuccessActivity.class);  
    startActivity(i);  
}
```

MainActivity (GET – example)

```
        else {
            mHandler.post(new Runnable() {
                public void run() {
                    Toast.makeText(MainActivity.this,
                                   "Wrong ID or Password",
                                   Toast.LENGTH_LONG).show();
                    et2.setText("");
                }
            });
        }

        input.close();
    } catch (Exception e) {
        Log.e("Net", "Error", e);
    }
}
}
```

SuccessActivity (GET – example)

```
package my.edu.utar.networkproject;

import android.os.Bundle;
import android.widget.*;
import android.app.Activity;

public class SuccessActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        RadioGroup rg = new RadioGroup(this);
        RadioButton rb1 = new RadioButton(this);
        RadioButton rb2 = new RadioButton(this);
        RadioButton rb3 = new RadioButton(this);
        RadioButton rb4 = new RadioButton(this);
        RadioButton rb5 = new RadioButton(this);
```

SuccessActivity (GET – example)

```
rb1.setText("New");  
rb2.setText("Edit");  
rb3.setText("Save");  
rb4.setText("Close");  
rb5.setText("Exit");
```

```
rg.addView(rb1);  
rg.addView(rb2);  
rg.addView(rb3);  
rg.addView(rb4);  
rg.addView(rb5);
```

```
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(rg);  
setContentView(ll);
```

```
}
```

```
}
```

Post Method

- Posting a form is a little more complicated than the default get method.
- The parameters had to be sent out through an **OutputStream** rather than directly appending to the url
- The **URLConnection** can be set to be post method through setting the output (**setDoOutput**)
- The request property shall also be set
- The **InputStream** should be return by the **URLConnection** instead

MainActivity2 (POST – example)

```
public void run() {  
    try {  
        String parameters = "name="+name+"&password="+pwd;  
        URL url = new URL("http://10.0.2.2:8085/ServletTest/LoginPost");  
  
        Log.i("Net",url.toString());  
  
        URLConnection hc = (URLConnection)url.openConnection();  
        hc.setDoOutput(true);  
        hc.setRequestProperty("Content-Type",  
            "application/x-www-form-urlencoded");  
  
        OutputStream output = hc.getOutputStream();  
        output.write(parameters.getBytes());  
        output.flush();  
  
        int length = hc.getContentLength();  
    }  
}
```

MainActivity2 (POST – example)

```
InputStream input = hc.getInputStream() ;

byte[] binput = new byte[length];
int readSize = input.read(binput);

Log.i("Net","readSize = " + readSize);
Log.i("Net","bText = "+ new String(binput));

String result = new String(binput).trim();

if (result.equals("OK")){
    Intent i = new Intent(MainActivity2.this,
        SuccessActivity.class);
    startActivity(i);
}
```

MainActivity2 (POST – example)

```
        else {
            mHandler.post(new Runnable() {
                public void run() {
                    Toast.makeText(MainActivity2.this,
                                   "Wrong ID or Password",
                                   Toast.LENGTH_LONG).show();
                    et2.setText("");
                }
            });
        }

        input.close();
        output.close();
    } catch (Exception e) {
        Log.e("Net", "Error", e);
    }
}
```

Public Web Services

- Here is a list of free and open public APIs:
 - Agify.io – Predicts age based on a name
 - JokeAPI – Jokes (<https://sv443.net/jokeapi/v2/>)
 - Listly – Get Metadata for a URL by scraping the page content (e.g. <https://list.ly/api/v4/meta?url=http://google.com>)
 - Nationalize.io – Predict nationality based on a name
 - Universities List – Universities (e.g. <http://universities.hipolabs.com/search?country=Malaysia>)

Source: <https://mixedanalytics.com/blog/list-actually-free-open-no-auth-needed-apis/>

SSL Check Bypass

- For setting up an all-trusting SSL context to bypass SSL certificate checks on Android

```
private static void setupATSSLContext(SSLContext sc) {  
    try {  
        // Create a trust manager that does not validate  
        certificate chains  
        TrustManager[] trustAllCerts = new TrustManager[] {  
            new X509TrustManager() {  
                @Override  
                public void  
                checkClientTrusted(X509Certificate[] chain, String authType) throws  
                CertificateException {  
                    // No-op: accept all client certificates  
                }  
            }  
        }  
    }  
}
```

SSL Check Bypass

```
        @Override
        public void
checkServerTrusted(X509Certificate[] chain, String authType) throws
CertificateException {
            // No-op: accept all server certificates
        }

        @Override
        public X509Certificate[] getAcceptedIssuers()
{
            return new X509Certificate[0];
            // No accepted issuers
        }
    }
};
```

SSL Check Bypass

```
// Install the all-trusting trust manager
sc.init(null, trustAllCerts, new
java.security.SecureRandom());

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

- To use it, initialise and setup such security context before making HTTPS connection to the web server.

```
SSLContext sc = SSLContext.getInstance("TLS");
setupATSSLContext(sc);
HttpsURLConnection hc =
    (HttpsURLConnection) url.openConnection();
hc.setSSLSocketFactory(sc.getSocketFactory());
```

Supabase (Create new project)

Create a new project

Your project will have its own dedicated instance and full postgres database.
An API will be set up so you can easily interact with your new database.

Organization

tanchiangkang's Org

Name

Practical 8

Database Password

.....

Copy

This password is strong. [Generate a password](#)

Region

🇸🇬 Southeast Asia (Singapore)

Select a region close to your users for the best performance.

Pricing Plan

Free - \$0/month

Select a plan that suits your needs. [More details](#)

Cancel

You can rename your project later

Create new project

Supabase (API Settings)

API Settings

Project URL

URL

`https://vwlasacnimdvjupmvsne.supabase.co`

Copy

A RESTful endpoint for querying and managing your database.

Project API keys

Your API is secured behind an API gateway which requires an API Key for every request.
You can use the keys below to use Supabase client libraries.

anon

public

`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlbiI6I`

Copy

This key is safe to use in a browser if you have enabled Row Level Security for your tables and configured policies.

Note that your API settings will be different from the above.

Supabase (Create Table)

- Create an arbitrary table
- Disable Row Level Security (for now)
- id and created_at columns are included by default
- Add columns for
 - Name (not nullable)
 - Age
 - Programme

The image shows the Supabase 'Create Table' interface. The table name is 'Students' and the description is 'Optional'. The 'Enable Row Level Security (RLS)' checkbox is unchecked, with a warning that turning it off allows anonymous access. The 'Enable Realtime' checkbox is also unchecked. The 'Columns' section lists five columns: 'id' (int8, primary key, nullable), 'created_at' (timestamp, nullable, default 'now()'), 'Name' (text, nullable), 'Age' (int2, nullable), and 'Programme' (text, nullable).

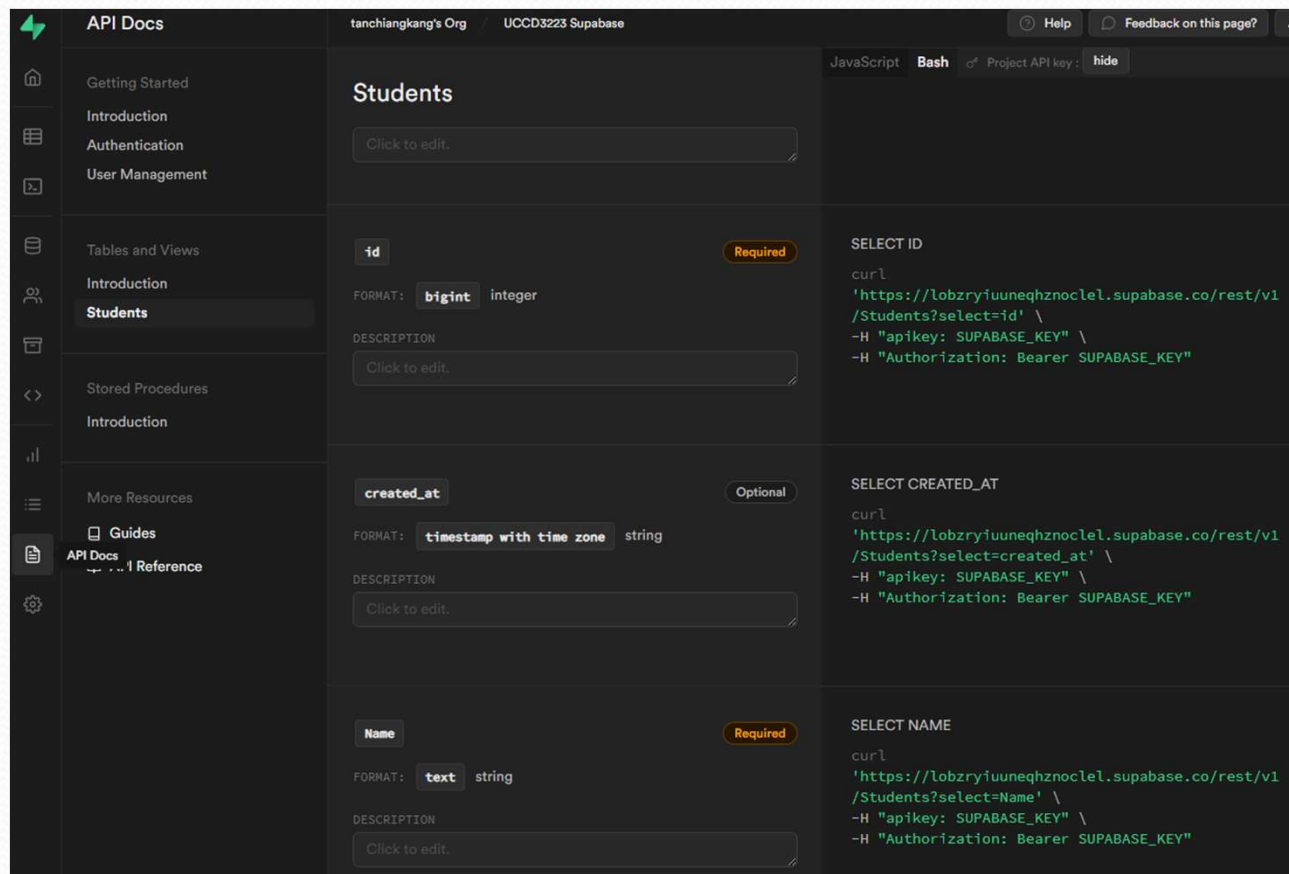
Name	Type	Default Value	Primary
id	# int8	NULL	✓
created_at	📅 timestampz	now()	☐
Name	T text	NULL	☐
Age	# int2	NULL	☐
Programme	T text	NULL	☐

Supabase (API Access)

- API key
 - Only application that has the API key included can have access to the database.
 - The key can be stored in a resource file (e.g. res/values/secrets.xml) in Android project
 - Key can be accessed like normal strings
- // inside of an Activity, `getString` is called directly
- ```
String secretValue = getString(R.string.SUPABASE_KEY);
```
- // if inside an XML file
- ```
@string/SUPABASE_KEY
```

Supabase (API Access)

- Table can be accessed via REST API (refer to API Docs)



The screenshot displays the Supabase API Docs interface for a project named 'UCCD3223 Supabase'. The left sidebar contains a navigation menu with sections: 'Getting Started' (Introduction, Authentication, User Management), 'Tables and Views' (Introduction, **Students**), 'Stored Procedures' (Introduction), and 'More Resources' (Guides, API Docs, Reference). The main content area is titled 'Students' and shows the table's schema and REST API endpoints.

Students Table Schema:

- id** (Required): Format: **bigint** integer. Description: Click to edit.
- created_at** (Optional): Format: **timestamp with time zone** string. Description: Click to edit.
- Name** (Required): Format: **text** string. Description: Click to edit.

REST API Endpoints:

- SELECT ID**:
curl
'https://lobzryiuuueqhznoclel.supabase.co/rest/v1/Students?select=id' \
-H "apikey: SUPABASE_KEY" \
-H "Authorization: Bearer SUPABASE_KEY"
- SELECT CREATED_AT**:
curl
'https://lobzryiuuueqhznoclel.supabase.co/rest/v1/Students?select=created_at' \
-H "apikey: SUPABASE_KEY" \
-H "Authorization: Bearer SUPABASE_KEY"
- SELECT NAME**:
curl
'https://lobzryiuuueqhznoclel.supabase.co/rest/v1/Students?select=Name' \
-H "apikey: SUPABASE_KEY" \
-H "Authorization: Bearer SUPABASE_KEY"

Supabase (API Access)

- Queries to access table can be carried out via HTTP GET request
- Inserting data into the table can be done using HTTP POST request
- Updating data can be done using HTTP PATCH request method
- Deleting data can be done using HTTP DELETE request method

[Remember to refer to API Docs to find out what HTTP request properties need to be set too]